

Recurrent Neural Networks

Introduction to Sequence Models

Applications of Sequence Models

Sequence models, particularly Recurrent Neural Networks (RNNs) and their variations, have transformed fields like speech recognition, natural language processing, and molecular biology. These problems are framed as supervised learning tasks that map an input X to an output label Y . Because the data points follow a specific ordering over time or space, traditional feedforward networks are insufficient, making specialized sequence architectures necessary.

The flexibility of sequence models allows them to be applied across diverse domains with different data modalities:

- **Speech Recognition:** Maps an input audio clip X over time to a textual sequence of words Y .
- **Music Generation:** Takes a minimal input X , such as a single integer for a genre or an empty set $\emptyset$, and generates a sequence of notes Y .
- **Sentiment Classification:** Analyzes an input text sequence X (e.g., a movie review) to predict a star rating or sentiment score Y .
- **DNA Sequence Analysis:** Processes a biological sequence X composed of the alphabets $\{A, C, G, T\}$ to label which parts correspond to specific proteins Y .
- **Machine Translation:** Takes an entire sentence X in a source language and translates it into a sequence Y in a target language.
- **Video Activity Recognition:** Looks at a sequence of successive video frames X to identify the activity Y being performed.
- **Named Entity Recognition:** Considers an input sentence X and outputs a sequence of labels Y to identify entities like people, organizations, or locations.

Variations in Sequence Types

A defining characteristic of sequence problems is the structural relationship between the input X and the output Y . Unlike standard supervised learning where inputs and outputs have fixed dimensions, sequence problems display varied mapping architectures depending on whether the input, output, or both are sequences.

When both X and Y are sequences, their lengths do not necessarily have to match. For instance, in named entity recognition, the input sequence length matches the output sequence length ($T_x = T_y$). However, in machine translation, the source and target sentences typically have different lengths ($T_x \neq T_y$). There are also asymmetrical configurations: sentiment classification maps a sequence input to a single scalar or categorical output (many-to-one),

whereas music generation transforms a static input into an extended sequence output (one-to-many).

Key Terms/Formulas:

- Recurrent Neural Network (RNN): A class of neural networks designed to model sequential data by passing hidden states across time steps.
 - Input Sequence (X): The ordered set of features fed into a model, which may represent time series, text, or audio data.
 - Output Sequence (Y): The target or predicted sequence generated by the model.
 - Sequence Length (T): The total number of time steps or tokens contained within a given sequence, often denoted as T_x for inputs and T_y for outputs.
-

Notation for Sequence Models

Sequence Notation and Motivating Example

To establish a standardized mathematical framework for sequence models, consider the task of Named-Entity Recognition (NER). Given an input sentence such as "Harry Potter and Hermione Granger invented a new spell", the goal is to automatically detect which words correspond to specific entities, such as people's names. The target output sequence Y assigns a binary or categorical label to each word indicating whether it belongs to a target entity. For this specific sequence of nine words, the model maps a sequence of inputs directly to a sequence of outputs.

The formal notation tracks individual elements across sequence positions (often representing temporal steps) and across multiple training instances:

- The t -th sequential element of an input sequence is represented as $X^{(t)}$.
- The corresponding target output at that specific position is represented as $Y^{(t)}$.
- The total length of the input sequence is denoted by T_x , while the length of the output sequence is denoted by T_y . For this specific NER example, $T_x = T_y = 9$.
- To refer to the t -th element of the i -th training example, the notation combines a parenthetical superscript with an angle-bracket superscript, written as $X^{(i)(t)}$ for inputs and $Y^{(i)(t)}$ for outputs.
- The length of the input sequence for the i -th training example is written as $T_x^{(i)}$, accounting for the fact that different sentences in a training set can have varying lengths.

Word Representation and Vocabularies

Natural Language Processing (NLP) requires a structured method to transform raw text into numerical vectors that machine learning architectures can process. The first step involves building a vocabulary (or dictionary), which is a discrete, sorted list of all unique words

considered by the model. While commercial systems typically utilize vocabulary sizes ranging from 30,000 to over 1,000,000 words, experimental setups often restrict this to a manageable size, such as 10,000 of the most frequent words found in the training corpus.

Once the vocabulary is established, each word is transformed into a high-dimensional sparse vector using a one-hot representation. The dimensionality of this vector is equal to the size of the vocabulary. Every position in the vector contains a 0 except for the single index corresponding to that word's sorted position in the dictionary, which contains a 1.

For an input word vector $X^{(t)}$ mapped to a vocabulary index k , the mathematical layout is structured as a column vector:

$$X^{(t)} = \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$$

If a word encountered during testing or training does not exist within the predefined vocabulary, it cannot be mapped to a standard index. To address this, a special fallback token known as the unknown word token, denoted as $\langle \text{UNK} \rangle$, is included in the vocabulary to represent all out-of-vocabulary terms uniformly.

Key Terms/Formulas:

- **Named-Entity Recognition (NER):** A sequence labeling task aimed at identifying and classifying entities (e.g., people, locations, dates) within text.
 - **Sequence Index (t):** An indicator used to reference the specific position or time step within a sequence, denoted as $X^{(t)}$ or $Y^{(t)}$.
 - **Sequence Length (T_x, T_y):** Variables representing the total number of tokens or time steps in the input and output sequences, respectively.
 - **Training Example Index ((i)):** A superscript notation used to specify the i -th training sample, resulting in notations like $X^{(i)(t)}$ and $T_x^{(i)}$.
 - **Vocabulary:** A defined dictionary of unique words used to construct word representations; its size determines the dimensionality of the input vectors.
 - **One-Hot Vector:** A sparse representation method where a word is represented by a vector of vocabulary size containing exactly one 1 at its corresponding index and zeros elsewhere.
 - **Unknown Word Token ($\langle \text{UNK} \rangle$):** A unique token designated to represent any out-of-vocabulary word encountered by the sequence model.
-

Recurrent Neural Network Model

Limitations of Standard Neural Networks

Standard feedforward neural networks are poorly suited for sequence processing due to two main architectural drawbacks. First, inputs and outputs can vary in length across different training examples, meaning $T_x^{(i)} \neq T_x^{(j)}$ for different samples i and j . While zero-padding sequences to a fixed maximum length can act as a workaround, it does not provide a robust or efficient representation. Second, a naive network fails to share features learned across different positions of the text sequence. If the network learns that the word "Harry" at position 1 signifies a person's name, it cannot automatically generalize that discovery to another position $X^{(t)}$.

Furthermore, utilizing large input representations across fixed structures leads to an explosion of parameters. If each input word is represented by a 10,000-dimensional one-hot vector, a standard first layer weight matrix would scale to an enormous, unmanageable size.

Unidirectional RNN Architecture

A Recurrent Neural Network (RNN) resolves these limitations by scanning through data sequentially from left to right while sharing a consistent set of parameters across all time steps. At time step t , the network ingests the current sequence element $X^{(t)}$ alongside an activation hidden state passed from the previous step $a^{(t-1)}$. It uses this coupled information to compute the current hidden state $a^{(t)}$ and predict the current time-step output $\hat{Y}^{(t)}$. To initiate this system at $t = 1$, a baseline activation vector $a^{(0)}$ is required, which is standardly initialized as a vector of zeros, though random initialization is sometimes utilized.

While some literature represents RNNs compactly using a looped cell configuration with a designated time-delay indicator block, mentally unrolling the network across successive time steps provides a clearer representation of data flow. A critical weakness of this basic unidirectional structure is that the prediction at any step $\hat{Y}^{(t)}$ relies solely on historical context from $X^{(1)}$ to $X^{(t)}$, completely omitting future elements. For example, in the sentences "He said Teddy Roosevelt was a great president" and "He said teddy bears are on sale", a unidirectional model cannot accurately classify the word "Teddy" at step 3 without evaluating the subsequent tokens. This constraint is resolved by Bi-directional RNNs (BRNNs).

Forward Propagation and Simplified Notation

The forward propagation phase calculates the internal hidden states and output predictions sequentially across the timeline. The basic calculations map connections to explicit weight parameters:

$$a^{(t)} = g(W_{aa}a^{(t-1)} + W_{ax}X^{(t)} + b_a)$$

$$\hat{Y}^{(t)} = g(W_{ya}a^{(t)} + b_y)$$

Within these equations, g represents the activation function. Typically, a tanh or a ReLU function is applied to compute hidden state activations, whereas the secondary activation function for the output layer depends entirely on the target label configuration (e.g., a sigmoid function for binary classification or a softmax function for K -way classification).

To simplify notation when constructing complex networks, these parameter equations can be compressed by horizontally stacking the weight matrices and vertically stacking the feature vectors. We define a unified weight matrix $W_a = [W_{aa} \mid W_{ax}]$ by placing the matrices side-by-side. If the hidden layer $a^{(t)}$ has a dimension of 100 and the input $X^{(t)}$ has a dimension of 10,000, then W_{aa} is a 100×100 matrix, W_{ax} is a $100 \times 10,000$ matrix, and the consolidated matrix W_a has dimensions of $100 \times 10,100$. The input vectors are stacked vertically to form a combined vector of size 10,100:

$$\begin{bmatrix} a^{(t-1)} \\ X^{(t)} \end{bmatrix}$$

Using this unified convention, the simplified forward propagation equations are written as:

$$a^{(t)} = g \left(W_a \begin{bmatrix} a^{(t-1)} \\ X^{(t)} \end{bmatrix} + b_a \right)$$

$$\hat{Y}^{(t)} = g(W_y a^{(t)} + b_y)$$

Key Terms/Formulas:

- **Parameter Sharing:** The technique of reusing the same structural weight matrices (W_{aa} , W_{ax} , W_{ya}) across all sequential time steps to ensure generalization across sequence locations and limit overall model size.
 - **Hidden State ($a^{(t)}$):** The internal memory vector computed at time step t that summarizes information from past sequence elements, calculated via $a^{(t)} = g(W_{aa}a^{(t-1)} + W_{ax}X^{(t)} + b_a)$.
 - **Unified Parameter Matrix (W_a):** A horizontally concatenated parameter matrix defined as $W_a = [W_{aa} \mid W_{ax}]$, used to simplify mathematical representation by processing stacked activation and input vectors simultaneously.
 - **Unidirectional RNN:** An RNN architecture that processes sequences strictly forward in time, meaning predictions at step t cannot leverage context from future indices.
-

Backpropagation in Recurrent Neural Networks

Computation Graph and Forward Pass

Before computing gradients, an RNN constructs a sequential computation graph via a forward propagation pass executed from left to right. Given an input sequence $X^{(1)}, X^{(2)}, \dots, X^{(T_x)}$, the network utilizes a shared set of parameters W_a and b_a to compute the sequential hidden layer activations $a^{(t)}$. At each individual time step t , these activations are subsequently mapped to predictions $\hat{Y}^{(t)}$ using another shared set of parameters W_y and b_y . Because these parameters are shared across every step, any parameter update must account for its accumulated influence across the entire temporal span of the sequence.

Loss Function Framework

To evaluate the accuracy of the predictions, an element-wise loss function must be defined at each individual sequence position. For binary classification tasks like named-entity recognition, the model employs the standard cross-entropy loss (logistic regression loss) for a single time step t :

$$\mathcal{L}^{(t)}(\hat{Y}^{(t)}, Y^{(t)}) = -Y^{(t)} \log \hat{Y}^{(t)} - (1 - Y^{(t)}) \log(1 - \hat{Y}^{(t)})$$

The global error metric is the accumulated sequence loss, which provides a single scalar summarizing performance across all tokens. Assuming the input sequence length equals the output sequence length ($T_x = T_y$), the overall loss $\mathcal{L}$ for the entire sequence is computed by summing the individual step losses across all time steps:

$$\mathcal{L} = \sum_{t=1}^{T_x} \mathcal{L}^{(t)}(\hat{Y}^{(t)}, Y^{(t)})$$

Backpropagation Through Time (BPTT)

The parameter optimization phase requires carrying out derivative calculations in the exact opposite direction of the forward propagation pathways. Gradients flow from the final consolidated loss $\mathcal{L}$ back into the individual step losses $\mathcal{L}^{(t)}$, up through the predictions $\hat{Y}^{(t)}$, and recursively backwards from right to left through the hidden state activations $a^{(t)}$.

This recursive calculation going backwards through sequential steps gives the algorithm its distinct name:

- **Direction of Flow:** While forward propagation increases the time index t from left to right, backpropagation decreases t , effectively flowing backwards through time.
- **Parameter Updates:** The resulting partial derivatives are aggregated across all time steps to update the shared parameter matrices W_a and W_y , along with biases b_a and b_y , via gradient descent.
- **Automated Frameworks:** In practice, modern deep learning libraries automatically manage this computation graph and handle the tracking of temporal dependencies during backpropagation.

Key Terms/Formulas:

- **Element-wise Loss ($\mathcal{L}^{(t)}$):** The discrepancy between the predicted value $\hat{Y}^{(t)}$ and actual label $Y^{(t)}$ at a specific sequence position t , defined as $\mathcal{L}^{(t)}(\hat{Y}^{(t)}, Y^{(t)}) = -Y^{(t)} \log \hat{Y}^{(t)} - (1 - Y^{(t)}) \log(1 - \hat{Y}^{(t)})$.
 - **Overall Loss ($\mathcal{L}$):** The total loss of a sequence instance, defined mathematically as $\mathcal{L} = \sum_{t=1}^{T_x} \mathcal{L}^{(t)}$.
 - **Backpropagation Through Time (BPTT):** The specific implementation of backpropagation applied to recurrent architectures, where gradients are propagated sequentially backwards from the final time step to the initial time step.
 - **Shared Parameters (W_a, b_a, W_y, b_y):** The core weights and biases that receive gradient accumulations from every temporal step during the BPTT process.
-

Different RNN Architectures

Overview of Sequence Typologies

Standard RNN applications do not always require the length of the input sequence T_x to equal the length of the output sequence T_y . Depending on the specific machine learning task, these sequences can display asymmetrical lengths or even behave as single vectors rather than sequences. Inspired by Andrej Karpathy's analysis of sequence modeling, these structural permutations can be divided into distinct mapping categories tailored to specific natural language processing and generation tasks.

Structural Categories of RNNs

The classification of recurrent network topologies depends heavily on the dimensional relationship between the inputs and outputs:

- **One-to-One:** This configuration handles a single input X to generate a single output Y . It acts as a standard feedforward neural network and does not leverage the temporal memory mechanisms found in sequence modeling.
- **Many-to-One:** The network takes an input sequence of length T_x and processes it step-by-step, but generates an output prediction Y only at the final time step. This structure is commonly used for sentiment classification, where an entire sentence of text is read before outputting a categorical star rating or binary sentiment score.
- **One-to-Many:** This architecture maps a single input X to an output sequence of length T_y . It is primarily applied in sequence generation tasks such as music generation, where a genre seed or a null vector input triggers a sequence of individual notes. In practice, the generated output from a previous time step $\hat{Y}^{(t-1)}$ is passed forward to serve as the input for the current time step t .
- **Many-to-Many ($T_x = T_y$):** This represents the classic recurrent network structure

where an input sequence and an output sequence have identical lengths. At each time step t , the input $X^{(t)}$ updates the internal hidden state and immediately produces a corresponding prediction $\hat{Y}^{(t)}$, which is the primary paradigm for Named-Entity Recognition (NER).

- **Many-to-Many ($T_x \neq T_y$):** When sequence lengths diverge, such as in machine translation where a source sentence and its translation have different token counts, an unaligned many-to-many structure is required. This setup splits the network into an encoder and a decoder. The encoder processes the incoming stream of text to build a contextual vector, and the decoder then takes over to emit the target sequence.

Key Terms/Formulas:

- **Many-to-One Architecture:** An RNN setup that ingests a sequence of features over T_x steps but returns a single prediction vector or scalar at the final step.
- **One-to-Many Architecture:** An RNN variant designed to transform a single static input X into a sequence of values spanning T_y steps.
- **Encoder-Decoder Framework:** An asymmetrical many-to-many RNN architecture where an encoder processes an input sequence of length T_x into a latent state, and a decoder reads that state to generate an output sequence of length T_y .

Language Modeling with an RNN

The Role of Language Modeling

Language modeling is a fundamental pillar of natural language processing that evaluates the likelihood of a sequence of words. In applications like speech recognition or machine translation, it acts as a critical discriminator between phrases that sound identical but differ radically in semantic plausibility. For instance, a speech system relies on a language model to determine that the phrase "the apple and pear salad was delicious" possesses a significantly higher probability of occurrence than "the apple and pair salad". Mathematically, given an arbitrary sentence represented as a sequence of tokens $Y^{(1)}, Y^{(2)}, \dots, Y^{(T_y)}$, the model's objective is to compute the joint probability $P(Y^{(1)}, Y^{(2)}, \dots, Y^{(T_y)})$.

Data Preprocessing and Tokenization

Before feeding raw text into an RNN, a text corpus must undergo tokenization to map individual words to discrete indices within a predefined vocabulary. During this step, specialized tokens can be injected into the dictionary to handle structural boundaries and exceptions:

- **End of Sentence ($\langle \text{EOS} \rangle$):** This token can be appended to the end of every sentence in the training set, allowing the model to explicitly learn and predict when a sentence is likely to terminate.
- **Unknown Word ($\langle \text{UNK} \rangle$):** When an input phrase contains out-of-vocabulary words that

do not match the top frequent words (such as a 10,000-word vocabulary limit), they are uniformly replaced by this token to allow probability estimation.

- **Punctuation Handling:** Depending on the application, punctuation marks (such as periods or commas) can either be filtered out entirely or treated as distinct tokens with their own dedicated positions in the vocabulary matrix.

RNN Structure for Word Prediction

To model the sequential dependency of language, the RNN architecture sets its input at time step t to be equal to the actual target word from the previous step, establishing the relationship $X^{(t)} = Y^{(t-1)}$. At the initial step $t = 1$, both the baseline activation $a^{(0)}$ and the initial input vector $X^{(1)}$ are established as vectors of zeros. The hidden state $a^{(1)}$ is then used to compute a high-dimensional Softmax output $\hat{Y}^{(1)}$, which defines a probability distribution over the entire vocabulary representing the likelihood of every possible initial word.

In subsequent steps, the ground-truth word from the previous time step is fed as the current feature vector. For example, at $t = 2$, the network receives $X^{(2)} = Y^{(1)}$ and outputs $\hat{Y}^{(2)}$, mapping the conditional probability of the second word given the first token. This sequential structure continues across the length of the text sequence, training the network to predict tokens step-by-step from left to right based on the cumulative historical activation context.

Loss Function and Probability Computation

Training a language model requires optimizing parameters by minimizing a global sequence error via gradient descent. At any individual time step t , the error is calculated using the categorical cross-entropy (Softmax loss) function:

$$\mathcal{L}^{(t)}(\hat{Y}^{(t)}, Y^{(t)}) = - \sum_{k=1}^V Y_k^{(t)} \log \hat{Y}_k^{(t)}$$

where V is the total vocabulary size. The total loss $\mathcal{L}$ across the entire sequence is computed as the sum of these isolated step metrics:

$$\mathcal{L} = \sum_{t=1}^{T_y} \mathcal{L}^{(t)}(\hat{Y}^{(t)}, Y^{(t)})$$

Once trained, the model evaluates the overall probability of any novel sentence by applying the chain rule of probability to multiply the successive conditional distributions generated at each Softmax layer:

$$P(Y^{(1)}, Y^{(2)}, \dots, Y^{(T_y)}) = P(Y^{(1)}) \times P(Y^{(2)} | Y^{(1)}) \times \dots \times P(Y^{(T_y)} | Y^{(1)}, \dots, Y^{(T_y-1)})$$

Key Terms/Formulas:

- **Corpus:** A large, structured body of text data utilized to train natural language processing models.
- **Tokenization:** The process of breaking down a raw sentence into individual structural

pieces (tokens), such as words or punctuation marks, and mapping them to numerical vectors.

- End-of-Sentence Token ($\langle \text{EOS} \rangle$): A unique token placed at the terminal boundary of a sentence to model sentence termination probabilities.
 - Softmax Loss ($\mathcal{L}^{\langle t \rangle}$): The step-wise categorical cross-entropy loss that evaluates the discrepancy between the predicted vocabulary distribution $\hat{Y}^{\langle t \rangle}$ and the true token vector $Y^{\langle t \rangle}$, given by $\mathcal{L}^{\langle t \rangle} = - \sum_{k=1}^V Y_k^{\langle t \rangle} \log \hat{Y}_k^{\langle t \rangle}$.
 - Joint Sequence Probability ($P(Y^{\langle 1 \rangle}, \dots, Y^{\langle T_y \rangle})$): The total likelihood of a sentence, calculated via the product of sequential conditional probabilities: $\prod_{t=1}^{T_y} P(Y^{\langle t \rangle} | Y^{\langle 1 \rangle}, \dots, Y^{\langle t-1 \rangle})$.
-

Sampling Novel Sequences

Sampling Novel Sequences from a Language Model

After training a sequence model, you can informally evaluate what the model has learned by generating or sampling completely novel sequences of text. While training requires feeding the true ground-truth words from the training set into successive time steps, the sampling process dynamically feeds the network's own predictions back into itself.

The procedure initializes identically to training, setting the initial hidden activation $a^{\langle 0 \rangle}$ and the initial time-step feature vector $X^{\langle 1 \rangle}$ to vectors of zeros:

$$X^{\langle 1 \rangle} = \vec{0}, \quad a^{\langle 0 \rangle} = \vec{0}$$

At time step $t = 1$, the model outputs a high-dimensional Softmax vector $\hat{Y}^{\langle 1 \rangle}$ containing probabilities for every token in the vocabulary. Rather than picking the absolute maximum probability (greedy token choice), a token is randomly selected according to the probability distribution defined by the Softmax vector, typically executed via functions like

`np.random.choice`.

This sampled token becomes the explicit input feature vector for the subsequent step, establishing the recurring relationship:

$$X^{\langle t \rangle} = \hat{Y}^{\langle t-1 \rangle}$$

At time step $t = 2$, the network uses $X^{\langle 2 \rangle}$ along with the hidden state $a^{\langle 1 \rangle}$ to produce a new distribution $\hat{Y}^{\langle 2 \rangle}$. This process repeats iteratively until a stopping criterion is achieved. The sequence generation can terminate in two primary ways:

- If an end-of-sentence token $\langle \text{EOS} \rangle$ is present in the vocabulary, generation stops immediately upon sampling it.
- Alternatively, the generation loop can be hard-coded to stop after a fixed threshold, such

as 20 or 100 time steps.

If the sampling algorithm accidentally selects an unknown word token $\langle \text{UNK} \rangle$, you can either include it in the final string or discard the selection and resample from the remaining vocabulary until a valid, known token is pulled.

Word-Level vs. Character-Level Modeling

The structural unit of a language model depends on the configuration of its vocabulary. In a word-level RNN, the dictionary is composed of full structural words. Conversely, a character-level RNN utilizes an alphabet of individual characters (such as $\{a, b, c, \dots, z\}$, uppercase letters, spaces, numbers, and basic punctuation) as its foundational tokens. This shift changes the sequence definition: a sentence like "cats average" transforms from a two-token sequence into a twelve-token character sequence including the space.

Character-level language models introduce distinct structural tradeoffs:

- **Advantages:** They entirely eliminate the problem of dealing with unknown word tokens $\langle \text{UNK} \rangle$. Because the vocabulary covers all individual alphabetic characters, the model can assign non-zero probabilities to novel, out-of-vocabulary words encountered during inference.
- **Disadvantages:** The primary bottleneck is sequence length inflation. Splitting sentences into single characters results in significantly longer temporal dimensions. This structural extension makes it much harder for basic RNNs to capture long-range dependencies across text spans and significantly increases the computational cost of backward propagation over time.

Key Terms/Formulas:

- Random Choice Sampling: A technique where a sequence token is non-deterministically selected based on the probability distribution vector generated by a Softmax activation layer.
 - Output-to-Input Feeding ($X^{(t)} = \hat{Y}^{(t-1)}$): The core execution pattern of sequence generation where a sampled token from the previous time step serves as the input token for the current time step.
 - Character-Level RNN: A sequence model architecture whose underlying vocabulary is composed of single characters rather than complete words, avoiding out-of-vocabulary exceptions at the cost of increased sequence lengths.
 - Long-Range Dependencies: The contextual relationships between early parts of a sequence and later steps, which become more difficult to compute as the number of sequential tokens grows.
-

Vanishing Gradients in RNNs

The Challenge of Long-Term Dependencies

Language tasks often exhibit long-term dependencies where a token appearing early in a sequence directly governs grammatical choices much later. For example, in the sentence "The cat, which already ate a bunch of food..., was full", the singular subject "cat" dictates the choice of the singular verb "was" rather than the plural "were". A basic RNN struggles to maintain this distant contextual tracking because its structure behaves like a very deep network when unrolled over time. If an input sequence spans hundreds or thousands of steps ($T_x = 1000$), it effectively creates a 1000-layer deep computational graph, making early layers highly insulated from late-stage training feedback.

Vanishing Gradients in RNNs

The core limitation of the basic RNN architecture is its susceptibility to the vanishing gradient problem. During the backward pass of Backpropagation Through Time (BPTT), gradients must flow backwards through time from right to left. The error signal computed at a late output layer step, such as $\hat{Y}^{(t)}$, drops off exponentially as it is multiplied recursively by the shared weight parameters across successive steps.

Consequently, the gradient has a difficult time propagating all the way back to the earliest nodes in the computation graph. This creates a strong "local influence" bias: the predictions at a step like $\hat{Y}^{(3)}$ are overwhelmingly influenced by nearby input features rather than early context tokens. The model effectively forgets information that occurred far back in the history, preventing it from tracking long-range patterns.

Exploding Gradients and Gradient Clipping

While gradients can decrease exponentially, they can also grow exponentially as a function of the number of sequential layers, leading to exploding gradients. This phenomenon is highly catastrophic because the derivatives blow up, disrupting the weight configurations and resulting in numerical overflows often indicated by NaN values in training logs.

Gradient Problem	Diagnostic Symptoms	Common Resolution
Vanishing Gradients	Poor long-range tracking, frozen weights in early layers	Advanced Gated Architectures (GRU, LSTM)
Exploding Gradients	Catastrophic parameter updates, NaN values	Gradient Clipping

Fortunately, exploding gradients are relatively easy to identify and fix using a technique called gradient clipping. This optimization constraint scales down the gradient vector if its magnitude crosses a given ceiling:

Gradient Clipping Rule: If the norm of the gradient vector $\|g\|$ exceeds a predefined maximum threshold value c , the gradient is scaled back proportionally before updating the weights:

$$g \leftarrow c \frac{g}{\|g\|}$$

This relatively robust constraint prevents catastrophic updates and ensures stable training trajectories, though resolving the vanishing gradient issue requires more advanced gating architectures.

Key Terms/Formulas:

- **Long-Range Dependencies:** Relationships in sequential data where early elements influence late-stage predictions across a large temporal distance.
- **Vanishing Gradient:** The exponential decay of error gradients as they are propagated backward through deep networks or long temporal steps, causing early weights to stop updating effectively.
- **Exploding Gradient:** The exponential growth of derivatives through successive computational steps, causing severe numerical instability and parameter destruction.
- **NaN (Not a Number):** A numerical error value indicating floating-point overflow, frequently caused by exploding gradients in unclipped neural networks.
- **Gradient Clipping:** A regularization technique that limits the maximum magnitude of a gradient vector g by re-scaling it if its norm $\|g\|$ surpasses a designated threshold c .

Gated Recurrent Unit (GRU)

Introduction to Gated Recurrent Units

Standard recurrent neural networks struggle significantly with the vanishing gradient problem, preventing them from capturing long-range semantic dependencies. For example, in sentences where a singular subject ("the cat") is separated from its verb ("was") by a long string of modifying clauses, a baseline RNN often fails to maintain the grammatical context over long distances. The Gated Recurrent Unit (GRU) alters the structural composition of the hidden layer to solve this issue. It introduces a specialized tracking variable called a memory cell, denoted as $c^{(t)}$, which provides a dedicated channel for preserving historical context across extensive sequential steps. In a basic GRU configuration, the output activation state $a^{(t)}$ is exactly equivalent to the internal cell state $c^{(t)}$.

Simplified GRU Mechanics and Gating

To control when information is maintained, modified, or dropped, the simplified GRU structure computes a candidate state value $\tilde{c}^{(t)}$ along with an internal update gate represented by the uppercase Greek letter Γ_u . The update gate passes its inputs through a sigmoid activation function σ , outputting values bounded tightly between 0 and 1. Intuitively, this gate acts like a binary switch: if $\Gamma_u \approx 1$, the network actively overwrites the current memory cell with the new candidate context; if $\Gamma_u \approx 0$, the network ignores the current input and perfectly retains the old memory cell value $c^{(t-1)}$ forward through time.

The forward mathematical calculations that govern a simplified GRU at step t are organized as follows:

$$\tilde{c}^{(t)} = \tanh \left(W_c \begin{bmatrix} c^{(t-1)} \\ X^{(t)} \end{bmatrix} + b_c \right)$$

$$\Gamma_u = \sigma \left(W_u \begin{bmatrix} c^{(t-1)} \\ X^{(t)} \end{bmatrix} + b_u \right)$$

$$c^{(t)} = \Gamma_u * \tilde{c}^{(t)} + (1 - \Gamma_u) * c^{(t-1)}$$

When scaled to high-dimensional hidden layers (such as a 100-dimensional activation vector), the memory cells and gates match this dimensionality, and the asterisk symbol $*$ explicitly denotes element-wise multiplication. This vectorization allows the GRU to perform selective routing, choosing to update certain dimensional bits of context (like singular/plural flags) while preserving other unrelated dimensions of semantic history (like vocabulary themes) perfectly across thousands of time steps without experiencing gradient decay.

The Full GRU Architecture

The complete instantiation of a standard GRU extends the simplified system by adding a secondary gate called the relevance gate, denoted as Γ_r . This gate determines exactly how important or relevant the past historical cell state $c^{(t-1)}$ is to the calculation of the current step's prospective candidate memory $\tilde{c}^{(t)}$.

The combined set of equations defining a full, robust Gated Recurrent Unit is formalized below:

$$\Gamma_u = \sigma \left(W_u \begin{bmatrix} c^{(t-1)} \\ X^{(t)} \end{bmatrix} + b_u \right)$$

$$\Gamma_r = \sigma \left(W_r \begin{bmatrix} c^{(t-1)} \\ X^{(t)} \end{bmatrix} + b_r \right)$$

$$\tilde{c}^{(t)} = \tanh \left(W_c \begin{bmatrix} \Gamma_r * c^{(t-1)} \\ X^{(t)} \end{bmatrix} + b_c \right)$$

$$c^{\langle t \rangle} = \Gamma_u * \tilde{c}^{\langle t \rangle} + (1 - \Gamma_u) * c^{\langle t-1 \rangle}$$

$$a^{\langle t \rangle} = c^{\langle t \rangle}$$

Key Terms/Formulas:

- **Memory Cell ($c^{\langle t \rangle}$):** The internal state container of a gated recurrent node that preserves sequential context, matching the output activation vector $a^{\langle t \rangle}$ in GRU networks.
 - **Candidate Value ($\tilde{c}^{\langle t \rangle}$):** A prospective update vector calculated at time step t via a tanh layer, representing the newest contextual proposal to be evaluated by the gating mechanism.
 - **Update Gate (Γ_u):** A sigmoid-activated control vector calculated at each step that determines whether the memory cell adopts the new candidate state ($\Gamma_u \approx 1$) or holds onto historical values ($\Gamma_u \approx 0$).
 - **Relevance Gate (Γ_r):** An auxiliary gate in the full GRU formulation that scales the influence of the previous cell state $c^{\langle t-1 \rangle}$ when building the current candidate value $\tilde{c}^{\langle t \rangle}$.
 - **Element-wise Multiplication (*):** The Hadamard product operation applied to matrices and vectors of matching dimensions, controlling local context filtering across individual vector bits.
-

Long Short-Term Memory (LSTM)

LSTM Architecture and Gating Mechanisms

The Long Short-Term Memory (LSTM) network, originally proposed by Sepp Hochreiter and Jürgen Schmidhuber, is a powerful sequence modeling architecture designed to overcome the vanishing gradient problem. While the Gated Recurrent Unit (GRU) simplifies this structure, the LSTM offers a more general and flexible framework. A key architectural distinction in LSTMs is that the hidden state output activation $a^{\langle t \rangle}$ is no longer identical to the internal memory cell state $c^{\langle t \rangle}$. This decoupling allows the network to maintain its long-term memory via the cell state while fine-tuning the temporal outputs through the hidden state.

Instead of a single update gate controlling both the preservation and replacement of memory, the LSTM utilizes three independent gating mechanisms: the update gate Γ_u , the forget gate Γ_f , and the output gate Γ_o . All three gates utilize a sigmoid activation function σ , outputting values between 0 and 1 to regulate information flow.

The forward propagation equations governing the LSTM at time step t are calculated as follows:

$$\tilde{c}^{\langle t \rangle} = \tanh \left(W_c \begin{bmatrix} a^{\langle t-1 \rangle} \\ X^{\langle t \rangle} \end{bmatrix} + b_c \right)$$

$$\Gamma_u = \sigma \left(W_u \begin{bmatrix} a^{\langle t-1 \rangle} \\ X^{\langle t \rangle} \end{bmatrix} + b_u \right)$$

$$\Gamma_f = \sigma \left(W_f \begin{bmatrix} a^{\langle t-1 \rangle} \\ X^{\langle t \rangle} \end{bmatrix} + b_f \right)$$

$$\Gamma_o = \sigma \left(W_o \begin{bmatrix} a^{\langle t-1 \rangle} \\ X^{\langle t \rangle} \end{bmatrix} + b_o \right)$$

$$c^{\langle t \rangle} = \Gamma_u * \tilde{c}^{\langle t \rangle} + \Gamma_f * c^{\langle t-1 \rangle}$$

$$a^{\langle t \rangle} = \Gamma_o * \tanh(c^{\langle t \rangle})$$

By unrolling these units in parallel, the cell state creates a clear, uninterrupted horizontal pathway across time steps. If the forget gate is set to $\Gamma_f = 1$ and the update gate is set to $\Gamma_u = 0$, the network can propagate an early memory state (e.g., $c^{\langle 0 \rangle}$) directly across multiple temporal links such that $c^{\langle 3 \rangle} = c^{\langle 0 \rangle}$. This explains why LSTMs are exceptionally proficient at memorizing characteristics across hundreds of sequence steps.

Peephole Connections and Architectural Variations

Several structural variations of the standard LSTM exist in academic literature. The most common modification introduces peephole connections. In a standard LSTM cell, the three internal gating arrays depend entirely on the previous hidden state $a^{\langle t-1 \rangle}$ and the current step input $X^{\langle t \rangle}$. A peephole connection modifies this arrangement by allowing the gates to also inspect the historical memory cell state $c^{\langle t-1 \rangle}$ directly.

When incorporating peephole dependencies, the mathematical formulations for the gates are augmented to include the previous cell vector:

- The previous cell state $c^{\langle t-1 \rangle}$ is injected directly into the sigmoid operations for Γ_u , Γ_f , and Γ_o .
- This dependency operates via an element-wise mapping. For a 100-dimensional hidden memory cell vector, the k -th element of $c^{\langle t-1 \rangle}$ uniquely influences the k -th element of the gating vectors, rather than computing a full matrix cross-multiplication.

Comparative Analysis: LSTM vs. GRU

Choosing between a GRU and an LSTM depends heavily on the specific engineering constraints and data requirements of a machine learning task, as neither model is universally superior.

- **Gated Recurrent Unit (GRU):** The GRU features a simpler design utilizing only two gates (Γ_u and Γ_r). Because it contains fewer parameters, it is more computationally efficient, executes faster, and scales more easily when building exceptionally large deep learning models.

- **Long Short-Term Memory (LSTM):** The LSTM is more flexible and structurally expressive due to its three independent gates. Historically, the LSTM has served as the default, proven choice for sequence processing tasks.

Key Terms/Formulas:

- Long Short-Term Memory (LSTM): A gated sequence network architecture that tracks long-range context by separating its internal memory cell state $c^{\langle t \rangle}$ from its external hidden state activation $a^{\langle t \rangle}$.
 - Forget Gate (Γ_f): A sigmoid-activated control vector that decides which elements of the past memory cell state $c^{\langle t-1 \rangle}$ to discard, defined by $\Gamma_f = \sigma(W_f[a^{\langle t-1 \rangle}, X^{\langle t \rangle}] + b_f)$.
 - Output Gate (Γ_o): The gating mechanism determining what portions of the internal cell state $c^{\langle t \rangle}$ are passed to the visible output activation $a^{\langle t \rangle}$.
 - Peephole Connection: An LSTM variation allowing the internal gating architectures to incorporate the past cell state $c^{\langle t-1 \rangle}$ directly into their sigmoid logic via element-wise operations.
-

Bidirectional Recurrent Neural Networks

The Need for Future Context

Standard unidirectional recurrent neural networks process data strictly from left to right. A major limitation of this approach is that predictions at any given time step t are constructed using only the historical context from $X^{\langle 1 \rangle}$ through $X^{\langle t \rangle}$, ignoring all future sequence tokens. In tasks like Named-Entity Recognition (NER), this constraint hinders accuracy. For example, in the phrase "He said Teddy Roosevelt was a great president," evaluating the token "Teddy" at step 3 requires future tokens like "Roosevelt" to accurately distinguish the proper name from a phrase like "teddy bears". This limitation applies uniformly whether the hidden layer uses basic RNN nodes, GRUs, or LSTMs.

Bidirectional Architecture and Data Flow

A Bidirectional RNN (BRNN) addresses this problem by splitting the hidden layer into two independent computational paths: a forward recurrent component and a backward recurrent component. The forward path calculates activations sequentially from left to right, denoted as $a^{\rightarrow \langle t \rangle}$, starting at $t = 1$ and concluding at $t = T_x$. Conversely, the backward path computes activations from right to left, denoted as $a^{\leftarrow \langle t \rangle}$, initializing at $t = T_x$ and moving backward down to $t = 1$. It is important to clarify that both pathways are executed during the forward propagation phase of network execution, meaning the backward layers are generating activations rather than performing backpropagation derivatives.

Once both sequences of hidden state activations are computed across the full timeline, they are fed simultaneously into the output layer to form predictions. The formal computation for an output prediction $\hat{Y}^{(t)}$ at time step t relies on a joint parameter transformation of both contextual vectors:

$$\hat{Y}^{(t)} = g \left(W_y \begin{bmatrix} a^{\rightarrow(t)} \\ a^{\leftarrow(t)} \end{bmatrix} + b_y \right)$$

In this equation, W_y is the shared output weight matrix, b_y is the output bias vector, and g represents the final layer activation function. Because $\hat{Y}^{(t)}$ receives information from $a^{\rightarrow(t)}$ (which encapsulates past steps 1 to t) and $a^{\leftarrow(t)}$ (which encapsulates future steps T_x down to t), the network achieves a global awareness of the entire sequence context at every position.

Trade-offs and Blocks

The foundational nodes within a BRNN can be swapped out depending on the complexity of the processing task:

- **Cell Types:** The forward and backward tracking nodes can consist of baseline standard RNN blocks, GRUs, or LSTMs. For a vast majority of natural language processing problems, a bidirectional LSTM (BiLSTM) network serves as a highly robust and common baseline architecture.
- **Architectural Disadvantage:** The main bottleneck of a standard BRNN is that the network requires access to the entire data sequence before it can compute predictions at any single position.
- **Application Constraints:** This dependency makes BRNNs poorly suited for real-time applications such as live speech recognition, as the algorithm would force the system to wait for a speaker to complete their entire utterance before emitting any translated text. However, for core text-based NLP applications where a complete sentence or document is entirely available upfront, BRNN configurations are exceptionally effective.

Key Terms/Formulas:

- **Bidirectional RNN (BRNN):** A network topology that utilizes two separate hidden layers running in opposite directions to capture both past and future sequential information.
- **Forward Activation ($a^{\rightarrow(t)}$):** The internal hidden state array computed sequentially from the beginning to the end of a sequence.
- **Backward Activation ($a^{\leftarrow(t)}$):** The internal hidden state array computed sequentially from the terminal step back to the initial step of a sequence.
- **Output Prediction Formula ($\hat{Y}^{(t)}$):** The mathematical step combining past and future features to compute output values, given by $\hat{Y}^{(t)} = g(W_y[a^{\rightarrow(t)}; a^{\leftarrow(t)}] + b_y)$.
- **Bidirectional LSTM (BiLSTM):** A specific variant of a bidirectional sequence model that

utilizes Long Short-Term Memory blocks for both the forward and backward paths.

Deep Recurrent Neural Networks

Architecture of Deep RNNs

To learn highly complex functions from sequential data, multiple layers of recurrent units can be stacked on top of one another to form a Deep RNN. In this hierarchical setup, the raw sequence features $X^{(t)}$ serve as the direct input to the first hidden layer. For any subsequent layer l , the input at time step t is composed of the hidden state activation vector generated by the layer directly beneath it, $a^{[l-1](t)}$. To precisely track these activations across both spatial layers and temporal positions, the mathematical notation uses square brackets to indicate the layer index and angle brackets to indicate the time index, written as $a^{[l](t)}$.

Mathematical Formulation and Parameter Sharing

The computation of an internal activation state at layer l and time step t integrates information from two distinct directions: the horizontal activation from the previous time step within the same layer, $a^{[l](t-1)}$, and the vertical activation from the current time step in the preceding layer, $a^{[l-1](t)}$. For example, the mathematical step required to calculate the hidden state vector $a^{[2](3)}$ located at layer 2, time step 3 is structured as follows:

$$a^{[2](3)} = g \left(W_a^{[2]} \begin{bmatrix} a^{2} \\ a^{[1](3)} \end{bmatrix} + b_a^{[2]} \right)$$

In this system, g denotes the layer-specific activation function, while $W_a^{[2]}$ and $b_a^{[2]}$ represent the unified weight matrix and bias vector dedicated exclusively to the second layer. Every recurrent layer l maintains its own unique set of parameters $W_a^{[l]}$ and $b_a^{[l]}$. These parameters are shared horizontally across all sequence time steps t within that particular layer, but they are completely unshared vertically across different layer indices.

Depth Constraints and Structural Variations

While standard feedforward or convolutional networks regularly scale to over 100 hidden layers, deep recurrent architectures rarely exceed a small handful of temporal layers (typically capped around 2 or 3 layers). Because an RNN already unrolls extensively along the temporal dimension T_x , adding severe vertical depth creates a massive computational graph that becomes exceptionally expensive to train.

To introduce functional capacity without over-allocating temporal parameters, modern sequence configurations frequently implement alternative hybrid designs:

- **Stacked Recurrent Blocks:** A modest number of recurrent layers (such as a 3-layer base) process the data horizontally across time to capture structural long-range context.
- **Deep Feedforward Top:** The activations from the terminal recurrent layer are passed

vertically into a deep feedforward network that entirely lacks horizontal connections across time steps, which then generates the final output predictions $\hat{Y}^{(t)}$.

- **Cell and Directional Flexibility:** The foundational blocks within a deep configuration do not need to be simple RNN nodes; they can be fully swapped out for GRU or LSTM units. Additionally, these multi-layer networks can be stacked symmetrically to create Deep Bidirectional LSTMs.

Key Terms/Formulas:

- Deep Recurrent Neural Network (Deep RNN): A sequence network configuration constructed by stacking multiple recurrent hidden layers vertically, where the activations of layer $l - 1$ act as the input features for layer l .
- Layer Activation Notation ($a^{[l]\langle t \rangle}$): The standardized multi-index vector representing the network hidden state at layer l and temporal position t .
- Layer-Specific Parameters ($W_a^{[l]}, b_a^{[l]}$): The distinct weights and biases shared uniformly across time within layer l , utilized in the forward equation $a^{[l]\langle t \rangle} = g(W_a^{[l]}[a^{[l]\langle t-1 \rangle}; a^{[l-1]\langle t \rangle}] + b_a^{[l]})$.
- Hybrid Deep RNN: An optimized network configuration that stacks a few temporally connected recurrent layers underneath a series of deep feedforward layers to achieve expressive depth while minimizing sequential training bottlenecks.